FFNOML: technical reference

Implementation of the Python training and inference code, the GPU service and the FFNOInversion.jl synthesis and inversion package. Written from the source at commit `3cb9c14` on `main`, 4 November 2026. The physics is in `docs/physics/ffno-physics.pdf`.

Contents

1 System overview **3**

1.1 Two halves and one boundary . . . 3

1.2 Repository map . . . 3

1.3 Conventions that differ between the halves . . . 4

2 Python configuration **4**

3 Data **6**

3.1 Reading MULTI3D . . . 6

3.2 Features and targets . . . 6

3.3 Dataset construction: `build_dataset_ffno` . . . 6

3.4 Loaders . . . 6

4 The network **7**

4.1 Forward pass . . . 7

4.2 Block . . . 7

4.3 Spectral branch: `SpectralConv2dFull` . . . 7

4.4 Vertical branch: `BalancedVerticalPhysicsStack` . . . 7

4.5 Parameter counts . . . 8

4.6 Activation checkpointing . . . 8

5 Training **8**

5.1 Launch . . . 8

5.2 Construction . . . 8

5.3 Loss as actually configured . . . 8

5.4 The loop . . . 8

5.5 Checkpoints . . . 8

5.6 Resume and expansion . . . 9

6 Inference without the service **9**

6.1 Tiled: `-predict` . . . 9

6.2 Distributed full domain: `-fsdppredict` . . . 9

7 The GPU service **10**

7.1 Purpose . . . 10

7.2 Startup . . . 10

7.3 Wire protocol . . . 10

7.4 Evaluation . . . 10

7.5 Diagnostics . . . 11

8 FFNOInversion.jl: structure **11**
8.1 Dependencies . . . 11
8.2 Source files . . . 11
8.3 Run flow . . . 11
8.4 The factory . . . 12

9 Run contract and configuration **12**
9.1 Inputs . . . 12
9.2 Outputs . . . 12
9.3 Configuration reference . . . 12

10 Equilibrium adapter **13**

11 Population mesh **14**

12 Synthesis **14**
12.1 Setup . . . 14
12.2 Opacity kernels . . . 14
12.3 Formal solver . . . 15
12.4 Output units . . . 15
12.5 Wavelengths . . . 15

13 Observation model and objective **15**
13.1 PSF . . . 15
13.2 Objective . . . 15
13.3 Regularization . . . 15
13.4 Controls . . . 16

14 Gradient **16**
14.1 Column response . . . 16

15 Optimizer **16**
15.1 Checkpoints . . . 16

16 Viewing angle implementation **17**

17 Parallel runtime **17**

18 Running **17**
18.1 Python . . . 17
18.2 Olivia . . . 17
18.3 eagle7 . . . 18
18.4 hyades . . . 18
18.5 Reproducing the FAL-C comparison . . . 18

19 Tests and validation **19**

20 UQ tooling **20**

21 Known issues found while writing this document **20**

22 Limits **21**

1. System overview

1.1 Two halves and one boundary

The repository has a Python half and a Julia half, and they meet at a checkpoint file and a socket.

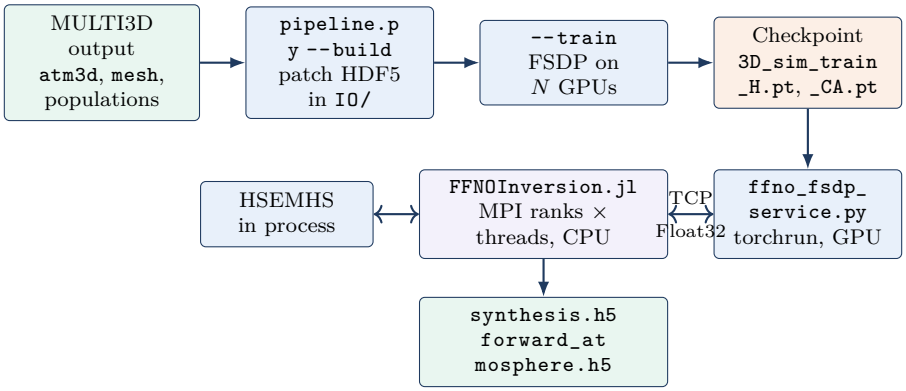


Figure 1: Components and their boundaries. Only Julia rank 0 talks to the service.

1.2 Repository map

Path	Role
`config.py`	Every Python setting: simulations, splits, atoms, model and training parameters
`pipeline.py`	Python command line: `--build`, `--train`, `--val`, `--test`, `--predict`, `--fsdppredict`, `--buildforpredict`, `--inspectfreqgate`
`FFNONet.py`	Dataset building, training driver, validation test, tiled and distributed prediction
`models/ffno_model.py`	The network
`model_builder.py`	Construction, FSDP or DDP wrapping, optimizer, scheduler, loss
`train_utils.py`	Training loop, checkpoints, resume, expansion
`data/dataset.py`, `data_builder.py`	Lazy HDF5 datasets and loaders
`loss/`	`nlte_composite_loss.py`, `weighted_mse_loss.py`, `gradient_loss.py`
`normalize_utils.py`	Channel statistics
`distributed_inference.py`	H-slab versions of FFT, GroupNorm and depthwise convolution
`ffno_fsdp_service.py`, `ffno_fsdp_runtime.py`, `ffno_fsdp_diagnostics.py`, `ffno_model_factory.py`	Persistent GPU service
`Forward.jl`	Older stand-alone synthesis from predicted HDF5 populations
`errorplots.py`, `make_paper_plots.py`, `compare_population_models.py`	Evaluation and figures
`scripts/`	Input builders (`build_input_ltau.py`, `build_input_logm.py`), Wittmann EOS (`witt_eos_cpp.cpp`, `witt.py`), UQ tools (`uq_*`)
`uq/`	UQ driver scripts; outputs are git-ignored
`FFNOInversion.jl/`	Julia package: `src/`, `ext/`, `scripts/`, `test/`, `docs/`
`examples/`	Run-directory templates: `inversion_run`, `python_run`, `runtime_test_run`
`response_run/`, `figures/`	Experiments and plotting scripts; data and images are not tracked

Path	Role
`tests/`	Python tests

1.3 Conventions that differ between the halves

	Python and network	Julia
Tensor layout	`[B, C, D, H, W]`, with D depth, $H = n_x$, $W = n_y$	atmosphere `(nz,nx,ny)`, populations `(nz,nx,ny,level)`, spectra `(wavelength,Stokes,x,y)`
Vertical order	As in the source; MULTI3D grids top first	Always top first: index 1 is the top; z decreases, $\log \tau$ and $\log m$ increase
Temperature	$\log_{10} T$	T [K]
Velocity	$\mathrm{km\,s^{-1}}$	$\mathrm{m\,s^{-1}}$
Height	Mm (`_normalize_z_scale`: $z/10^6$)	m
n_e, ρ	$\log_{10}$, $\mathrm{m^{-3}}$ and $\mathrm{kg\,m^{-3}}$	linear in the atmosphere; $\log_{10}$ in the feature cube
Output	normalized $\log_{10} n$	linear n [$\mathrm{m^{-3}}$]
Horizontal spacing	m	m

Table 2: The service converts between the two inside the differentiated forward pass, so VJP cotangents stay with respect to the SI quantities Julia sent.

2. Python configuration

`config.py` is imported with `from config import *`. Nothing is passed on the command line except the mode, so a run is defined by this file and the environment.

Name	Value	Meaning
DATA_DIR	EAGLE7_DIR	Root of bifrost_data; edit to OLIVIA_DIR on Olivia
RUN_DIR	env FFNOML_RUN_DIR	Disposable files: IO/, checkpoints. Defaults to the repository
ACTIVE_ATOMS	["H"]	env FFNOML_ACTIVE_ATOMS, comma separated
MODEL_DIR	training_FFNO3D_zscale_expand_lognlte	Under RUN_DIR
PATCH, STRIDE, SCALES	40, 20, 1–8	Patch extraction
NUM_EPOCHS	400	env FFNOML_NUM_EPOCHS
BATCH_SIZE	1	Required: depth varies between simulations
LEARNING_RATE, MIN_LEARNING_RATE	10^{-3}, 10^{-8}	Cosine schedule endpoints
WEIGHT_DECAY, GRAD_CLIP	3×10^{-4}, 1.0	AdamW; gradient norm clip
PATIENCE, MIN_DELTA	50, 5×10^{-5}	Early stopping
TRAINSELECT, TRAINSELECT_SEED	1, 12345	Fraction of patches

ATOM_CONFIG gives, per species, the MULTI3D output subdirectory, line level pairs and wavelengths [Å], level energies chi [J] for levels above the ground and the level count. It is only needed for the loss's source-function term.

MODEL_CONFIG is in_channels=6, out_channels=6, width=48, n_layers=4, dropout=0, spec_dropout=vertical_dropout=0.08 in all four layers, hidden_spectral=hidden_vertical=128, checkpoint_blocks=True. MODEL_CONFIG must match the checkpoint: ffno_model_factory.create_ffno3d builds from it, and the service loads with strict=True.

3. Data

3.1 Reading MULTI3D

pipeline.load_multi3d_blocks uses helita.sim.multi3d. Units are converted on read:

```
lte = m3d.atom.nstar * 1e6;  nlte = m3d.atom.n * 1e6   # cm-3 -> m-3
rho = atmos.rho * 1e3;       ne = atmos.ne * 1e6       # g cm-3 -> kg m-3, cm-3 -> m-3
vx, vy, vz = atmos.v*                                  # km/s, unchanged
z_scale = m3d.geometry.z * 1e-2                        # cm -> m
dx, dy = median spacing of mesh x, y, cm -> m
```

read_mesh parses the whitespace-separated mesh file (nx, x..., ny, y..., nz, z...). Array layout from helita is [nx, ny, nz, nlev].

3.2 Features and targets

_make_inputs_ch_first stacks $(\log_{10} T, v_x, v_y, v_z, \log_{10} n_e, \log_{10} \rho)$ to [Cin, nz, nx, ny]. _make_targets_ch_first discards LTE and returns $\log_{10} \max(n, 10^{-30})$. _make_log_lte_ch_first keeps $\log_{10} n^*$, unnormalized, solely for the source term of the loss.

3.3 Dataset construction: build_dataset_ffno

Two passes over all snapshots of the split:

1. Accumulate global sums in float64 and compute per-channel means and standard deviations. For the validation set, stat_file reuses the training statistics.
2. For each snapshot: normalize, then for each scale s apply gaussian_filter with $\sigma = (0, 0, s/2, s/2)$ and subsample by s, extract patches, and multiply dx, dy by s. z is subsampled but not filtered, and log LTE is filtered and subsampled. Scales whose grid is smaller than the patch are skipped.

Each simulation forms its own HDF5 group because depth differs. For a 256-pixel box: 121, 25, 9, 4, 1, 1 patches for $s = 1, \ldots, 6$, 161 in total, or 805 for five simulations. Each file is about 20 GB per species.

Item	Content
train_1/inputs, targets	[N, 6, D, 40, 40], normalized float32
train_1/log_lte_populations	[N, 6, D, 40, 40], unnormalized
train_1/z_scale	[N, D, 40, 40], Mm, unnormalized
train_1/dx, dy, scale, weights	per patch
mean_X, std_X, mean_Y, std_Y	root datasets
patch_dataset_names	group list
File attributes	Cin, Cout, N, patch, stride, scales, normalized, n_patch_datasets, target_representation = log10_nlte_population_m-3, lte_representation = log10_lte_population_m-3

Table 4: Patch file schema. The name is 3D_sim_<split>_<sims>_atoms<A>_target-lognlte_patch40_stride20.hdf5.

weights are the inverse scale count, normalized to mean 1. See Section 21 for why they currently have no effect.

3.4 Loaders

H5PatchDataset opens the file lazily in each worker and returns (x, y, z, dx, dy, scale, weight, log_lte). It reads both flat legacy and grouped layouts. With train_select < 1 it selects, per weight class, a fraction of patches and reassigns weights so that each class total is conserved; resample_train_dataset

redraws at each epoch with seed derived from the epoch. `DataLoaderBuilder` rejects `batch_size != 1` for variable depth, uses `DistributedSampler` when distributed, and turns off persistent workers for dynamic lengths.

`build_solving_set_ffno` writes an unnormalized single cube for prediction (`_save_hdf5_cube`). Normalization for prediction is taken from the checkpoint.

4. The network

4.1 Forward pass

`FFNO3D.forward(x, z_scale, dx, dy)`, with x `[B,6,D,H,W]` normalized and z `[B,D,H,W]` in Mm:

```
x = GELU(GroupNorm(Conv3d(6 -> 48, kernel 1, no bias)(x)))    # lift
for block in 4 x FFNOBlock3dBalanced: x = block(x, z, dx, dy)
y = Conv3d(96 -> 6, kernel 1)(GELU(Conv3d(48 -> 96, kernel 1, no bias)(x)))    # head
```

`_gn(c)` is `GroupNorm` with the largest group count ≤ 8 dividing c. Over 48 channels this is 8 groups. GroupNorm statistics cover the whole input volume, *so the normalization itself makes every output depend on the whole box and its size.*

4.2 Block

```
spec = spec_drop(GELU(norm_spec(SpectralConv2dFull(x, dx, dy))))
vert = vert_drop(GELU(norm_vert(BalancedVerticalPhysicsStack(x, z))))
g_s, g_v = chunk(branch_gate(cat[x, spec, vert]))              # sigmoid, 2*48 channels
s, v = spec*g_s, vert*g_v
fused = GELU(norm_fuse(fuse(cat[s, v]) + 0.5*(s + v)))
x1 = x + res_fused * fused                                     # init 1.0
x2 = x1 + res_pw * norm_pw(pw(x1))                             # init 0.15
out = x2 + res_mlp * tanh(norm_mlp(mlp(x2)))                   # init 0.15
```

`branch_gate`: Conv3d($3w \to w$, 1), GELU, Conv3d($w \to 2w$, 1, bias), Sigmoid. `fuse`: Conv3d($2w \to 2w$, 1), GELU, depthwise Conv3d($2w$, kernel 3, padding 1), GELU, Conv3d($2w \to w$, 1). `pw` and `mlp` are two-layer kernel-1 convolutions with expansion 2. Both are pointwise. With `collect_stats` or a `branch_mask` the block returns diagnostics or supports ablation.

4.3 Spectral branch: `SpectralConv2dFull`

```
x = x * input_gate(x)                          # pointwise sigmoid gate
X = rfft2(x, dim=(-2,-1))                      # all modes kept, on every depth plane
ky = fftfreq(H, d=dy) * 1e5;  kx = rfftfreq(W, d=dx) * 1e5
g = freq_mlp([kx, ky, sqrt(kx^2+ky^2+1e-12), sign(ky)])   # 4 -> 128 -> 128 -> 2, complex gate
Y_o = sum_i X_i * (W_real + i W_imag)_io * (g_r + i g_i)  # 48 x 48 complex matrix, shared over all k and depths
y = post(irfft2(Y, s=(H, W)))                  # Conv3d 48 -> 96 -> 48, kernel 1
```

Frequencies are in m^{-1}, scaled by 10^5, so the map applies at any grid size. No modes are truncated. Note that the same 48×48 matrix is modulated by one scalar gate per $\mathbf{k}$: every channel pair sees the same $\mathbf{k}$ dependence.

4.4 Vertical branch: `BalancedVerticalPhysicsStack`

Columns are reshaped to `[B*chunk, C, D]` and processed in chunks of 4 columns to bound memory.

```
z_feat = cat[z, (z-zmin)/span, |dz|/mean|dz|, span]   # with span clamped; dz central differences
y = in_proj(x) + z_proj(z_feat)                       # both to 128 channels
y = LiteMultiScaleVertical(kernels 3,7,15)(y)
y = LiteVerticalResidualBlock(k=5,d=1)(y); (k=5,d=2); (k=5,d=4); (k=7,d=1)
y = y * depth_gate(y)
y = out_gn(out_proj(y))                               # back to 48 channels
```

All convolutions are depthwise-separable 1D with zero padding at the top and bottom, with hidden multiplier 2. The filters act on grid index, not on physical z.

4.5 Parameter counts

Component (one block)	Parameters	Component (one block)	Parameters
Spectral branch	35 842	`branch_gate`	11 616
Vertical branch	697 056	`fuse`	16 416
`pw`	9 216	`mlp`	9 216
Whole model (4 blocks, lift, head)			**3 124 954**

Table 5: Counted by instantiating `FFNO3D(**MODEL_CONFIG)`. The vertical branch holds most of the weights.

4.6 Activation checkpointing

With `checkpoint_blocks=True`, lift, head and each block use non-reentrant `torch.utils.checkpoint` when gradients are enabled and stats are not collected. The switch is gradient mode, not train or eval mode, so inversion VJPs in eval mode also checkpoint. When the service wraps blocks itself, `external_block_checkpointing` disables the inner call for blocks.

5. Training

5.1 Launch

`train_gpu.sh` requests 8 nodes with 4 GPUs each on the Olivia `accel` partition, `mem-per-gpu=40G`, and runs torchrun. `examples/python_run` is the run directory template.

5.2 Construction

`ModelBuilder.build`: `init_process_group("nccl")`; `FFNO3D` on the local device; FSDP with `transformer_auto_wrap_policy` on `FFNOBlock3dBalanced`, so each block is one sharded unit; `AdamW`; `CosineAnnealingLR(T_max=NUM_EPOCHS, eta_min)`; `NLTECompositeLoss` over `WeightedMSE_L1`.

With env `FFNOML_TRAIN_ONLY=proj1,proj2`, other top-level modules are frozen and DDP replaces FSDP, because FSDP makes every wrapped unit's output require gradients, so frozen blocks would still be recomputed. This is the head-only fine-tuning of the UQ work.

5.3 Loss as actually configured

`NLTECompositeLoss` takes `lam=0, lam_S=0, lam_g=0` by default and `ModelBuilder` passes none of them:

$$L = \underbrace{\tfrac{1}{2}\langle(1+|y|)(\hat{y}-y)^2\rangle + \tfrac{1}{2}\langle|\hat{y}-y|\rangle}_{\texttt{WeightedMSE_L1}} + 0\cdot L_S + 0\cdot L_g. \tag{1}$$

The source term is still evaluated on every step for logging. It compares x_{lu} between prediction and truth, where x_{lu} is built from de-normalized $\log n - \log n^*$ and $\Delta E/kT$, T being de-normalized from channel 0. It therefore costs compute without influencing the weights. `logb` in that file is a historical name for $\log_{10} n$.

5.4 The loop

`train_one_epoch`: forward over the patch, then flatten to columns `(B*Ny*Nx, L, Nz)` for the loss, backward, clip (`model.clip_grad_norm_` under FSDP), step. After each epoch: scheduler step, validation, checkpoints. Early stopping: an improvement needs `best_val - val > MIN_DELTA`, and training stops when `epochs_no_improve > PATIENCE`. There is no AMP.

5.5 Checkpoints

`save_checkpoint_fsdp` gathers the full state with `get_model_state_dict(full_state_dict=True, cpu_offload=True)`. All ranks call it; rank 0 writes.

File	Content
`MODEL_FILE`, e.g. `3D_sim_train_H.pt`	Best validation: `epoch`, `model_state`, `opt_state`, `train_loss`, `val_loss`, `train_components`, `val_components`, `normalization_stats`, `io_metadata`
Resume file (`get_resume_checkpoint_path`)	Latest state, with scheduler and `completed_epochs`, `current_lr`, `training_stats`

Production checkpoint	Epoch	Train loss	Validation loss
`3D_sim_train_H.pt`	171	4.32×10^{-3}	5.88×10^{-3}
`3D_sim_train_CA.pt`	252	3.59×10^{-3}	7.31×10^{-3}

5.6 Resume and expansion

- `--train --resume`: from the resume file, with optimizer and scheduler.
- `--train --resume --bestpath`: from the best file with fresh optimizer state, with `RESUME_LAST_LR`, `RESUME_LAST_EPOCH`, and a validation baseline first.
- `--train --expand`: `expand_model_from_checkpoint` copies matching tensors, zero-pads `vertical.z_proj.0.weight` when it grew from 1 to 4 geometry channels, and sets `res_*` of new blocks to zero so the expanded model starts out equal to the old one. `MODEL_FILE` must be a new path.
- `LOAD_EARLIER_VAL=False` discards the earlier best validation value.
- `--val`: one validation pass, printing loss terms.

`load_checkpoint` tries `weights_only=True` with a NumPy allowlist, then falls back to a full unpickle with a warning. Only use trusted files.

6. Inference without the service

6.1 Tiled: `-predict`

`_predict_tiled`: tiles of side `patch` with stride `stride`, blended with a 2D Hann window clamped at 10^{-3}. NaN and Inf in a tile are zeroed with a warning. This path is memory-light but is a different operator from the full domain, because of GroupNorm statistics and the zero-padded convolution: 40^2 tiles differ from the full domain by a median of 0.064 dex, 192^2 tiles by 0.003–0.009 dex with p99 0.03–0.05 dex.

6.2 Distributed full domain: `-fsdppredict`

`ffno_predict_populations_distributed_full` splits H into slabs by `partition_range`, and `enable_distributed_inference` replaces modules in place, with weights shared:

Module	Distributed behaviour
`SpectralConv2dFull`	`rfft` over W; all-to-all so that each rank owns a slab of W-frequencies with all of H; `fft` over H; the spectral multiply on local frequencies; reverse transforms and all-to-all. Needs $\lfloor n_y/2 \rfloor + 1 \geq$ world size.
`nn.GroupNorm`	`DistributedGroupNorm`: all-reduce of the sum, sum of squares and count, reproducing whole-volume statistics. Differentiable.
Depthwise `Conv3d`, kernel 3	`DistributedHDepthwiseConv3d`: one-cell halo exchange in H (a custom autograd function with a reversed exchange as its VJP), with zeros at the outer edges, matching single-GPU zero padding.
`BalancedVerticalPhysicsStack`	Local columns, wrapped for progress reports.
Fusion at inference	`chunked_fuse_branches`: over depth in chunks of 32 with a one-cell depth halo, because the concatenations exceed T4 memory at 384^3.

`_linear_collective` applies a collective to the primal and tangent separately, which makes these layers usable in forward-mode AD as well.

7. The GPU service

7.1 Purpose

A checkpoint load and FSDP setup per evaluation would dominate the cost of an inversion. One torchrun job therefore keeps H and Ca loaded for the entire run and answers requests, and `PROTOCOL_VERSION` is 2.

7.2 Startup

1. Rank 0 of the Julia job writes a TOML manifest, `ffno-fsdp-service-<pid>.toml`, with the models, port and a token (`<pid>-<time_ns>`). **The manifest holds the token: do not commit these files.**
2. It starts the launcher: `scripts/ffno_fsdp_service.sh` (Slurm, nested `srun --overlap`, static rendezvous, node rank from hostname order) or `ffno_fsdp_service_local.sh` (one host, `--standalone`, `FFNO_FSDP_NPROC`, used on eagle7).
3. Each rank initializes NCCL, plus a CPU Gloo group for command broadcast, so idle GPU ranks block in a socket rather than spin. Only torchrun rank 0 reads each checkpoint. Other ranks receive states by FSDP's `sync_module_states` broadcast.
4. The composition is `FSDP(checkpoint(block))`, with `FULL_SHARD`, `use_orig_params`, `limit_all_gathers`. Wrapping the other way round is unsupported.
5. Rank 0 listens and accepts one client. Handshake: `HELLO <token>`, reply `READY 2 <world> <models> FULL_SHARD_H_SLAB`.

7.3 Wire protocol

Commands are one text line; arrays are raw little-endian Float32 in Fortran order.

Command	Client sends after the line	Service replies
`PREDICT name nz nx ny levels dx dy`	features `(6,nz,nx,ny)`, z `(nz,nx,ny)`	`OK PREDICT ...`, populations `(nz,nx,ny,levels)`
`VJP name nz nx ny levels dx dy`	features, z, cotangent `(nz,nx,ny,levels)`	`OK VJP ...`, feature cotangent, z cotangent
`JVP name nz nx ny levels dx dy`	features, z, tangent `(6,nz,nx,ny)`	`OK JVP ...`, population tangent
`SHUTDOWN`		`BYE`

Features are $(T, v_x, v_y, v_z, \log_{10} n_e, \log_{10} \rho)$ in SI. Rank 0 scatters along n_x using matched NCCL P2P. Constraint: $n_x \geq$ world size.

7.4 Evaluation

```
features, z_model = cat[log10(x[:,0:1]), x[:,1:4]/1e3, x[:,4:6]], z/1e6
pred = model((features - mean_x)/std_x, z_model, dx, dy)
populations = 10 ** (pred*std_y + mean_y)         # linear m^-3
```

Normalization comes from the checkpoint. The service checks inputs for finiteness and positive temperature, and outputs for positivity.

- **VJP.** `torch.autograd.backward` with the cotangent, then read leaf gradients. `autograd.grad` restricted to inputs is deliberately not used, because it can prune FSDP's parameter accumulators, skip its post-backward reshard hooks and corrupt later calls.
- **JVP.** Dual tensors under `no_grad` with activation checkpointing bypassed (`_without_activation_checkpointing`). Heights stay fixed.

7.5 Diagnostics

Variable	Effect
`FFNO_FSDP_DIAGNOSTICS=1,2`	Per-rank logs: versions, libraries, input statistics, memory, block progress. 2 adds every submodule. Slow.
`FFNO_FSDP_CORE_DUMPS=1`	Raises the core limit and gives each rank its own directory
`FFNO_FSDP_TRACEBACK_INTERVAL`	Periodic stack dumps. Leave unset. In job 2622023 the watchdog walking frames without the GIL caused SIGSEGV in every rank (Python 3.12.3, torch dynamo eval-frame shim)
`FFNO_POPULATION_MODEL_ORDER=H,CA`	Order of species requests

8. FFNOInversion.jl: structure

8.1 Dependencies

Julia 1.12 (manifest resolved with 1.12.3). `Project.toml` pins by Git revision: HSEMHS `29f2310` (`harshmathur1990/1DHSE3dMHS`, subdirectory `HSEMHS`) and Muspel `01ec68d` (`harshmathur1990/Muspel.jl`). Muspel is a weak dependency, loaded through the extension `ext/FFNOInversionMuspelExt.jl`. The local Muspel checkout must match on Olivia.

8.2 Source files

File	Contents
`Types.jl`	`VerticalAxis`, `ViewGeometry`, `Grid3D`, `Atmosphere3D`, `MagneticField3D`, `SpectralCube`
`ViewAngle.jl`	`los_velocity`, `path_factor`, `ShearPlan`, `shear_levels!` and transpose, view attributes and checks
`Parallel.jl`	`ParallelContext`, `Tile2D`, scatter and gather, `exchange_halos`, `RootGPUCoordinator`
`Diagnostics.jl`	GPU-control resource logs and failure records
`Nodes.jl`	`NodeField`, `expand_nodes`
`EOS.jl`	`WittmannEOS` adapter, `IdealGasEOS` test fixture
`ContinuumOpacity.jl`	Wittmann continuum; used for the test comparison and `KuruczLTEModel(include_continuum=true)`
`PopulationModels.jl`	Interface, `population_features`, recorded and mock models
`PopulationMesh.jl`	`MeshPopulationModel`, corona, deep extension, VJP, JVP
`LinePhysics.jl`, `Barklem.jl`	Line models, Kurucz parser, damping recipes; ABO tables in `data/barklem`
`Synthesis.jl`	`build_synthesis_setup`, formal solver, unit conversion
`Observations.jl`	`GaussianPSFObservation`, weights
`Regularization.jl`	Seven-slot vertical and horizontal penalties
`DistributedForward.jl`	`HybridForwardModel`, `forward!`, footprint state, distributed χ^2
`Equilibrium.jl`	`prepare_equilibrium`, `reconstruct_equilibrium!`, given-atmosphere backend
`FSDPService.jl`	Service client and launcher
`Solvers.jl`	Control maps, `DistributedInversionProblem`, `evaluate_objective!`, pattern-search prototype
`Gradients.jl`	Finite-difference oracle, Taylor and dot-product checks
`ProductionGradients.jl`	`HybridAdjointObjectiveGradient` and all VJPs
`ScalableSolvers.jl`	`lbfgs_invert!`
`IO.jl`	TOML parsing, `load_config`, dry-run summary, checkpoints
`Execution.jl`	HDF5 input and output, `run_inversion!`, `run_inversion_files!`

8.3 Run flow

```
scripts/invert.jl CONFIG.toml MODEL_FACTORY.jl
  run_inversion_files!(config_path, factory)
    load_config                      # validates everything it can
    initialize_parallel              # MPI THREAD_FUNNELED, BLAS and FFTW to 1 thread
    read_inversion_inputs on root    # axes reversed once to top first
    distribute_atmosphere            # x slabs under HSEMHS
    factory.build(config, distributed, workspace, context)
      WittmannEOS -> prepare_equilibrium -> launch_fsdp_population_models
      reconstruct_equilibrium! -> predict populations -> footprint_state!
      build_synthesis_setup -> HybridForwardModel
    forward: forward!(...) once          inversion: lbfgs_invert!(...)
    gather on root, write_inversion_outputs
    close_model_resources!
```

`forward!` runs on every rank in this order: equilibrium; populations; footprint state when needed; synthesis; observation; each stage printing `FORWARD stage=...`. The first objective evaluation reuses the equilibrium and populations prepared by the factory (`reuse_prepared`).

8.4 The factory

A file defining `FFNO_INVERSION_FACTORY::InversionModelFactory`, or returning one when included, keeps deployment details out of the science inputs. `examples/inversion_run/model_factory.jl` is the template. On error it calls `abort_on_rank_failure`, since otherwise collective cleanup would block on the ranks that failed. It also returns asset lists for the preflight. It loads H and Ca even when only one is used.

9. Run contract and configuration

9.1 Inputs

Dataset	Shape	Units	Notes
`logtau_500`, or `log10m`, or `z_m`	`(z)`	dex; dex; m	Native axis. `log10m`: $\mathrm{kg\,m^{-2}}$, or $\mathrm{g\,cm^{-2}}$ with attribute `units`
`x_m`, `y_m`	`(x)`, `(y)`	m	Required on z, uniform
`temperature`	`(time,z,y,x)` or `(z,y,x)`	K	
`vx`, `vy`, `vz`	as T	$\mathrm{m\,s^{-1}}$	`[atmosphere] vz_positive`
`vturb`	as T	$\mathrm{m\,s^{-1}}$	Optional
`Bx`, `By`, `Bz`	as T	T	Optional, all or none
`ne`	as T	$\mathrm{m^{-3}}$	Forward only
`hydrogen_populations`	`(level,z,y,x)`, optional time	$\mathrm{m^{-3}}$	Forward only, with `ne`; last level is protons

Table 7: Initial atmosphere. Names are mapped in `[atmosphere.datasets]`. `storage_order="python"` reverses row-major dimensions.

Observations: `intensity` and `sigma` as `(time,Stokes,wavelength,y,x)`, `wavelength_weights` as `(wavelength,Stokes)`, `spatial_weights` as `(y,x)`. They are in `[outputs] intensity_units` divided by the region `normalization`. There is no interpolation: the grids must match.

9.2 Outputs

Synthesis: `intensity`, `normalization`, `wavelength_m`, objective terms when inverting, and attributes `execution_mode`, `axis_order`, `wavelength_frame`, `provenance_toml`, `view_*`. Atmosphere: the native axis, `x_m`, `y_m`, all fields with a leading time axis, `populations/<species>` as `(level,z,y,x)`, and attributes `vertical_coordinate`, `vertical_unit`, `vertical_order`, `native_axis_dataset`, `solver_termination`. Files are overwritten.

9.3 Configuration reference

Key	Meaning and default
`mode`	`inversion` (default) or `forward`. `solver.max_iterations = 0` also selects forward
`[inputs]`	`initial_atmosphere_file`, `observation_file`, `time_index` (1-based)
`[outputs]`	`synthesis_file`, `atmosphere_file`; `intensity_units = si | cgs` (si); `wavelength_frame = air | vacuum` (air), a label only
`[atmosphere]`	`vertical_coordinate = log10tau500 | log10m | z`; `storage_order`; `vz_positive = up | down`; `[atmosphere.datasets]`
`[equilibrium]`	`backend = HSEMHS`; `model = column_hse` (`mhs` rejected); `gravity_m_s2` (274); `eos_table_file`; `on_nonconvergence`. Optional when `ne` and `hydrogen_populations` are given
`[equilibrium.column_ltau]`, `[equilibrium.column_logm]`	`align = ptot | none`
`[equilibrium.column_z]`	`anchor = bottom | top | height`, `P_ref_pa` (required), `z_ref_m`, `tau_top = exp_tail | half_cell | zero`, `n_sub`, `adapt`, `rk_tol`, `n_sub_max`, `H_max_m`, `mu`
`[grid]`	`dx_m`, `dy_m`
`[[regions]]`	`start_angstrom`, `step_angstrom`, `count` (always air), `normalization`, `psf_type = none | fpi | file`, `psf_file`
`[[regions.sources]]`	`mode = ffno`, the lines following from the region; or `mode = kurucz_lte` with `linelist_file`
`[population.mesh]`	`enabled` (true except on z), `bottom_m` (-10^6), `top_m` (1.4×10^7), `spacing_m` (2×10^4), `uniform_top_m` (4.5×10^6), `stretch` (1.01), `max_spacing_m` (10^5), `nabla` (0.13), `corona_temperature_k` (10^6 from a config), `corona_transition_m` (2.4×10^5)
`[view]`	`mu` (1), `azimuth_deg` (0), `geometry = plane_parallel | inclined`
`[physics]`	`redistribution = non_prd`; `line_center = reference | atom`; `continuum_populations` only `eos`
`[observation]`	`stokes = ["I"]`; `[observation.datasets]`; `[observation.gaussian_psf]` `spatial_fwhm_x_m`, `spatial_fwhm_y_m`, `spectral_fwhm_nm`. It applies to forward runs too
`[regularization]`	`horizontal_order`; `[regularization.vertical]` `types`, `regularize`, `weights`, `length_scale_dex`, `length_scale_m`; `[regularization.horizontal]`; `[regularization.scales]`
`[[inversion.controls]]`	`variable`, `log_tau_nodes` / `log_m_nodes` / `z_nodes_m`, `control_nx`, `control_ny`, `lower`, `upper`, `scale`
`[solver]`	`method = lbfgs`: `max_iterations` (50), `history_length` (10), `gradient_tolerance` (10^{-6}), `step_tolerance` (10^{-10}), `objective_tolerance` (10^{-12}), `initial_step` (1), `backtracking_factor` (0.5), `armijo_coefficient` (10^{-4}), `maximum_line_search_trials` (20), `maximum_forward_evaluations`, `curvature_tolerance` (10^{-10}), `checkpoint_every` (1), `checkpoint_path`. `method = prototype_pattern_search` for validation
`[parallel]`	`enabled`, `decomposition = x_slabs | cartesian_2d`, `threads_per_rank`, `min_columns_per_rank`, `gpu_launcher_rank`, `gpu_connect_timeout_seconds`, `gpu_status_timeout_seconds`, `gpu_diagnostic_interval_seconds`, `gpu_diagnostics_directory`

Unknown keys in `[equilibrium]` and `[population]` are errors. `scripts/dry_run.jl` prints the parsed configuration.

10. Equilibrium adapter

`Equilibrium.jl` holds the only route to P, ρ, n_e and the derived coordinate. HSEMHS is called in process through its embedding API.

- **Mode** is derived, not configured: `column_hse_z`, `column_hse_ltau`, `column_hse_logm`. Sections for another axis are rejected.
- **Prepare.** `prepare_equilibrium` is collective. Local failures (missing table, wrong grid, provenance mismatch) are agreed before any HSEMHS collective, through `HSEMHS.agree`. It rejects a table built with other partition functions or abundances than the synthesis Wittmann library.
- **Solve.** `reconstruct_equilibrium!` calls `HSEMHS.solve_embedded!`, fills fields, and leaves T, $\mathbf{B}$, $\mathbf{v}$, v_{turb} untouched. It returns a rank-identical `EquilibriumOutcome` instead of throwing for physics failures.
- **Layout.** Arrays convert between Julia `(z,x,y)` and HSEMHS `(z,y,x)`. HSEMHS solves on x slabs; `decomposition = x_slabs` is required, and the backend checks that its slab equals the rank's tile.
- **Given** n_{e} on log m uses `_given_ne_column_mass!`: P from `HSEMHS.lnP_from_lnne`, then `given_geometry!`. On other axes, `solve_embedded!(...; lnne)`.
- **Given atmosphere.** `set_given_state!` runs on reading; `GivenAtmosphereBackend` then only builds geometry, with no table and no MPI. Abundance constants: total 1.0950187898409673, mass per H $2.3396723599405787 \times 10^{-27}$ kg, from `scripts/witt_eos_cpp.cpp`.

The HSEMHS Wittmann table and the FFNO Wittmann library are separate builds from the same source. `scripts/validate_eos_consistency.jl` compares them.

11. Population mesh

`MeshPopulationModel` wraps any population model. Default mesh: 452 levels.

- **Forward.** `_mesh_features` per column, threaded. `_mesh_locate` classifies each mesh height as inside, above or below. `_mesh_corona!` and `_mesh_deep!` integrate $\ln P$ by RK4 with at most 20 km steps, with ρ, n_{e} from `HSEMHS.state`. `_mesh_lnP` is a Newton solve for $\ln P$ from $\ln \rho$ on the table. Above the table's low-pressure end the last state is held. Below, leaving the table is an error.
- **Inner call.** `_mesh_atmosphere` builds an `Atmosphere3D` on the mesh and calls the service.
- **Back.** Log-linear in z per point, with `_mesh_bracket` clamping at mesh ends.
- **Caching.** The last features and populations are cached; the VJP reuses them when features are unchanged.
- **VJP.** Back-map transpose, including a term in $\bar{z}$ from the height dependence, inner VJP, then column transpose. Finite differences cover the three top and bottom values of the extensions.
- **Inclined.** `_rays_to_cartesian_features` and `_cartesian_to_ray_populations` shift by $\mp \mathbf{t} z$, with $\ln T$ and $\ln n$ as interpolated variables. Adjoints use the exact transpose `_shear_log_transpose`, not the inverse shift.

Requires `eos_table_file`. The struct's own default `corona_temperature_k` is 0, but `load_config` sets 10^6 unless told otherwise.

12. Synthesis

12.1 Setup

`build_synthesis_setup` performs all file I/O once and returns one `RegionSynthesisSetup` per region, holding the air grid, the vacuum grid and a `MixedIntensitySynthesizer`. Contributors are in this order: one `MuspelContinuumModel`, then each source. A Kurucz selection keeps lines within a margin of $20 \times 2.5\,\mathrm{km\,s^{-1}}$ of the window. `_without_ffno_transitions` drops a line of the FFNO transition's own ion within 0.1 nm. An FFNO source names no line: `ffno_region_lines` takes every transition of the atom files (`ffno_atom_lines`) whose centre lies on the region's grid within the same margin, and is an error when there is none. Preflight calls it too, so it reports what production will use. Population levels are the atom's levels by increasing energy, as Muspel sorts them; the line index is the position in `radiative_bound_bound`. So Hα is line 5 with levels 2 and 3, and Ca II 854.2 is line 5 with levels 3 and 5. `FFNO_REFERENCE_NM` holds laboratory wavelengths matched by nearest wavelength, not by level.

12.2 Opacity kernels

Interface: `add_opacity_emissivity!(chi, eta, model, wavelength, atmosphere, x, y, populations)`, which adds into `(nz,nlambda)` arrays. `chi` is in $\mathrm{m^{-1}}$; `eta` is in $\mathrm{kW\,m^{-3}\,sr^{-1}\,nm^{-1}}$ in every model (`source_planck`), so intensities come out in $\mathrm{kW\,m^{-2}\,sr^{-1}\,nm^{-1}}$.

Muspel line (extension): Float32 kinematics, Voigt from `Muspel.create_voigt_itp` with $a \in [10^{-4}, 10]$ on 20 000 points and $v \in [0, 500]$ on 2 500 points; `BoundedVoigt` uses the exact erfcx outside the table, since Muspel's linear extrapolation goes negative. v uses `model.center_nm` for the centre and `line.lambda0` for the Doppler scale.

Continuum (`MuspelContinuumModel`): Muspel's NLTE continuum interpolant at the region's central vacuum wavelength on $\log_{10} T \in [3.2, 7.5]$ (216 points) and $\log_{10} n_\mathrm{e} \in [8, 24]$ (161 points), fixed so that it does not depend on the slab or trial temperatures. Atoms: Al, C, Ca, Fe, H_6, He, K I, Mg, N, Na, Ni I, O, S, Si, read once. Neutral H and protons from `eos_neutral_hydrogen!` and `eos_protons`.

Kurucz: C++ `witt_kurucz_populations` for the lower level and neutral H, Armstrong Voigt, $\gamma = \gamma_\mathrm{rad} + \gamma_\mathrm{Stark} n_\mathrm{e} + \gamma_\mathrm{vdW} n_\mathrm{H\,I}$. Zero K94 damping means "not given": $\gamma_\mathrm{rad} = A_{ul}$ and no Stark, as in RH. Van der Waals recipe in `KuruczLine.vdw_recipe`: `abo_explicit`, `abo_table`, `list`, `unsold`, or `none`.

12.3 Formal solver

`formal_solve!` (`ScalarFormalSolver`) goes from point `nz`, $I = \eta/\chi$, to point 1. Weights for a step Δ:

```
if dt > 40:      w1 = 0;  w2 = 1/dt;  w3 = 1 - w2
elif dt > 0.01:  w1 = exp(-dt);  u0 = (1-w1)/dt;  w2 = u0 - w1;  w3 = 1 - u0
else:            w1 = 1 - dt + dt^2/2;  w2 = (1/2 - dt/3)*dt;  w3 = (1/2 - dt/6)*dt
I = w1*I + w2*S_lower + w3*S_upper
```

Substeps: $m = \min(\lceil j \rceil, 256)$ where $j = \max(|\Delta \ln \chi|, |\Delta \ln \eta|) > 1$, with log-linear interpolation, when all values are positive.

`formal_column!` chooses the coordinate. On log τ (`_tau_integration`): $s = -\ln \tau_{500}$, with χ, η scaled in place by τ_{500}/χ_{500}. On log m (`_mass_integration`): $s = -\ln m$, scaled by m/ρ. Otherwise the path is z/μ. Columns are threaded with `:static` scheduling and per-thread workspaces.

12.4 Output units

`output_intensity_scale` converts: $I_\nu = I_\lambda \lambda^2 / c$ with vacuum λ, then to SI or cgs, then divides by `normalization`.

12.5 Wavelengths

`_air_to_vacuum_nm` is Edlén's form, identical to RH's `air_to_vacuum`, with no change below 200 nm. `cube.wavelength_m` is always air. `_output_wavelength` converts only on writing. RH's `vacuum_to_air` is a different formula, not the inverse: 3.1 mÅ different at Hα, 5.4 mÅ at 854.2 nm, 12.8 mÅ at 1565 nm. Comparisons at that level belong on vacuum axes.

13. Observation model and objective

13.1 PSF

`GaussianPSFObservation`: separable kernels with radius $\lceil 4\sigma \rceil$, re-normalized where truncated at grid edges, so boundaries are *not* periodic. The spectral part needs a uniform grid across the concatenated axis. Spatial convolution across tiles uses `exchange_halos`, and `_distributed_spatial_vjp` is its exact transpose. On slanted views FWHMs are stretched by $1/\mu$ along the tilt; only $\phi = 0, 90, 180, 270°$ are accepted with a spatial PSF.

13.2 Objective

$$J = \frac{1}{N_\mathrm{active}} \sum \left(w_{\lambda,s,x,y} \frac{I_\mathrm{syn} - I_\mathrm{obs}}{\sigma} \right)^2 + R, \qquad w = w_{\lambda s}\, w_{xy}. \tag{2}$$

Weights multiply the residual, so a weight enters squared. N_active counts non-zero weights.

13.3 Regularization

Vertical slots, in order: `temperature, vlos, vturb, B, inc, azi, pgas_boundary`. Types: 0 none; 1 first derivative; 2 deviation from depth mean; 3 deviation from zero; 4 second derivative. Temperature types 2 and 3 are forced to 0. Derivatives are with respect to native coordinate over `length_scale_dex` (default 1) or `length_scale_m` (no default on z; required for types 1 and 4). Term = `regularize`$\cdot w_i \cdot \langle(\cdot/\mathrm{scale})^2\rangle$. Horizontal: forward differences per metre, order 1 or 2, over uniquely owned differences (non-periodic). Note that with Δx in metres these terms are numerically small, so choose strengths accordingly. The total is rank-count independent.

13.4 Controls

`ControlMapSpec`: `(nnode, control_nx, control_ny)` values, with node positions in the grid's native kind (a mismatch throws), bounds, and scale. `expand_nodes` is trilinear on normalized horizontal coordinates and clamps beyond the end nodes. Each trial: restore immutable rank-owned reference (`_restore_trial_reference!`), then $A = A_{\mathrm{ref}} + \mathcal{I}(\mathbf{p} - \mathbf{p}_{\mathrm{initial}})$. `vlos` at $\mu < 1$ moves $\mathbf{v}$ along $\mathbf{n}$ only and can't be combined with Cartesian velocity controls. `_physical_controls_valid` checks the full-grid bounds and gives $J = \infty$ if violated. `refine_control_maps` starts a new stage, with fresh optimizer history.

14. Gradient

`HybridAdjointObjectiveGradient`, one reverse pass:

1. $\bar{I}_{\mathrm{out}} = 2w^2(I_{\mathrm{syn}} - I_{\mathrm{obs}})/(N\sigma^2)$.
2. `observation_vjp!`: PSF transpose with reverse halo exchange.
3. `synthesis_vjp!`: per region and column, recompute χ, η; `_formal_solve_vjp!` is the exact transpose including substeps; axis weights contribute to $\bar{\chi}_{500}$ or $\bar{\rho}$; each contributor's `add_opacity_emissivity_vjp!`.
4. `footprint_vjp!` when an inclined z view was used.
5. `predict_distributed_populations_vjp!`: mesh transpose and service VJP; features map back to $\bar{T}$, $\bar{\mathbf{v}}$, $\bar{n_{\mathrm{e}}}$, $\bar{\rho}$, $\bar{z}$.
6. `accumulate_control_vjp!`: transpose of node expansion for directly controlled fields.
7. `_force_regularization_vjp!`: for each scaled coordinate, centred finite difference of $\langle \mathrm{bar}, \mathrm{derived\ fields} \rangle + R$, second-order on unequal steps and one-sided at bounds. This costs **two equilibrium solves per control coordinate per gradient**, the main remaining cost. `force_balance_step` defaults to 10^{-4}.

Opacity pullbacks: Muspel by a centred difference in eight quantities $(T, n_{\mathrm{e}}, v_{\mathrm{los}}, n_{\mathrm{H\,I}}, n_{\mathrm{p}}, n_u, n_l, v_{\mathrm{turb}})$ at each cell against the cotangents, in `_cell_pairing`; continuum by local relative differences, including EOS hydrogen derivatives in T and P; Wittmann by `_local_wittmann_vjp!`, perturbing one level at a time, since the kernels are opaque C++.

A given atmosphere has no adjoint: the continuum VJP throws. Checks: `FiniteDifferenceObjectiveGradient`, `gradient_taylor_validation`, `dot_product_validation`. Measured at $\mu = 0.8$: 4.4×10^{-10} on $\log \tau$, 2.6×10^{-9} on z.

14.1 Column response

`scripts/column_response.jl` gives $R_v(\lambda, k; x, y) = \partial I / \partial v(k)$ with populations and hydrostatic structure fixed, one adjoint per wavelength. `scripts/ffno_population_tangents.jl` obtains $\dot{n}(k, x, y; k_0) = \sum_{x',y'} \partial n(k, x, y)/\partial T(k_0, x', y')$ by one JVP per depth seeded in every column; this is 55 GB for H on 504^2. With `--population-tangents`, `column_response.jl` adds the population response.

15. Optimizer

`lbfgs_invert!`, in scaled coordinates. Rank 0 forms directions and decides; all ranks evaluate.

1. Projected gradient: components pushing outward at an active bound are zeroed. Stop if its ∞-norm $\leq$ `gradient_tolerance`.
2. Two-loop recursion on the projected gradient, with $\gamma = s^{\mathsf{T}}y / y^{\mathsf{T}}y$ scaling. Fall back to projected steepest descent if not a descent direction. `_feasible_direction!` zeroes components that leave at active bounds.
3. Backtracking: `candidate = clamp(x + step*direction)`; accept on the Armijo condition with the actual projected displacement. Up to `maximum_line_search_trials`, halving.
4. Update: the pair is stored only if $s^{\mathsf{T}}y > \mathrm{tol}\,\|s\|\|y\|$. History length 10.
5. Terminations: `gradient_tolerance`, `step_tolerance`, `objective_tolerance`, `maximum_forward_evaluations`, line-search failure, iteration limit.

A rejected trial leaves mutable workspaces at that trial, so they are re-established before continuing. `write_lbfgs_diagnostics` writes the iteration record.

15.1 Checkpoints

`checkpoint!` serializes atomically. A file is bound to the capability manifest, layout signature, control mapping (configuration, baseline atmosphere checksum, node initialization, correction representation), and history length. An incompatible one is refused. Restart works across MPI and thread layouts, because only replicated controls and history are stored.

16. Viewing angle implementation

`ViewGeometry(mu, azimuth_deg, geometry, footprint)` lives on `Grid3D`. At $\mu = 1$, or with no `[view]`, every function is the identity and results are bitwise unchanged.

`ShearPlan(z, dx, dy, t, s)` holds per-level integer offsets and bilinear weights with $f = s\,t(z_k - z_{\mathrm{ref}})/\Delta$, $z_{\mathrm{ref}} = 0$, periodic. `shear_levels!` is threaded by y; `shear_levels_transpose!` is threaded by level, so writes do not collide. The opposite shift is not the inverse.

`needs_footprint`: inclined on z. Then `footprint_state!` gathers to root, shifts T, velocities, v_{turb} (log for positive fields) and populations, and re-scatters: one gather and one scatter per forward and per gradient. Native τ and m inputs need attributes `view_mu`, `view_azimuth_deg`, `view_geometry` (`check_input_view`). `warn_view_travel` warns when rays travel more than a box. `response_run/view_check/build_slanted_input.jl` builds slanted inputs from a cube.

17. Parallel runtime

- **MPI.** `THREAD_FUNNELED`: MPI calls only on the initializing thread (`_assert_mpi_thread`). Finalizes only what it initialized. `enabled=false` under a multi-rank launch is rejected.
- **Tiles.** `Tile2D`, `process_grid`. Numeric payloads move without Julia serialization. Halos are corner-complete.
- **Threads.** Julia threads over columns; BLAS and FFTW forced to 1. Set `OMP_NUM_THREADS=1`.
- **GPU control.** `RootGPUCoordinator`: a TCP status channel between ranks is set up *before* the GPU launch, so ranks don't sit in an MPI collective while Slurm and NCCL start the step. Non-root ranks have timeouts; `abort_on_rank_failure` aborts rather than deadlocks.
- **Species.** `CompositeDistributedPopulationModel`: gather features to root, one request per species, scatter results. Only root talks to the service.

Environment variable	Effect
`FFNO_GPU_CONNECT_TIMEOUT`, `FFNO_GPU_STATUS_TIMEOUT`	Override config timeouts
`FFNO_GPU_DIAGNOSTIC_INTERVAL`, `FFNO_GPU_DIAGNOSTICS_DIR`	Periodic resource logs
`FFNO_GPU_CONTROL_HOST`	Address advertised to ranks and service
`FFNO_FSDP_LAUNCHER`, `FFNO_FSDP_PYTHON`, `FFNO_FSDP_NPROC`	Local service launch
`FFNO_FSDP_SERVICE_PORT`, `FFNO_FSDP_SHUTDOWN_TIMEOUT`	Port and shutdown
`FFNO_FSDP_GPUS_PER_NODE`, `FFNO_FSDP_CPUS_PER_NODE`	Slurm service layout
`FFNO_THREADS_PER_RANK`	Threads per rank
`FFNO_INVERSION_CONFIG`, `FFNO_INVERSION_FACTORY`	Olivia submission
`FFNO_ATOM_DIR`	Atom directory in tests
`FFNOML_RUN_DIR`, `FFNOML_ACTIVE_ATOMS`, `FFNOML_NUM_EPOCHS`, `FFNOML_TRAIN_ONLY`	Python settings
`OLIVIA_JULIA_DEPOT`, `FFNO_PREFLIGHT_JULIA_DEPOT`	Julia depots

18. Running

18.1 Python

```
python pipeline.py --build                       # training and validation HDF5 in IO/
torchrun ... pipeline.py --train                 # or sbatch train_gpu.sh
python pipeline.py --buildforpredict --predname en024048_hion_385
python pipeline.py --predict --predname ...       # tiled
torchrun ... pipeline.py --fsdppredict --predname ...
```

Output: `output_3D_sim_s5_<NAME>_<MODEL>_<ATOMS>.hdf5`, with `nlte_populations` in m^{-3}, consumed by `Forward.jl`.

18.2 Olivia

Production uses Slurm partition `accel`, NVIDIA GPUs and the account `nn2834k`. By default: 8 nodes × 4 GPUs with 2 Julia ranks per node, 128 CPUs per task, `--network=job_vni`, `mem-per-gpu=40G`.

```
repo=/cluster/projects/nn2834k/harshm/FFNOML
run=/cluster/work/projects/nn2834k/harshm/ffnoml_forward_001
mkdir -p "$run"; cp -R "$repo/examples/inversion_run/." "$run/"
```

```
# link inputs, atoms, partition functions, EOS table, checkpoints
FFNOML_RUN_DIR="$run" FFNO_INVERSION_CONFIG="$run/forward.toml" \
  bash "$repo/FFNOInversion.jl/scripts/submit_olivia_inversion.sh"
```

Login and accelerator nodes differ in architecture (x86 and ARM neoverse_v2). Julia, Python and C++ differ between them; nothing built on one may run on the other, and `olivia_runtime_environment.sh` must never be sourced on a login node. The submission helper runs a CPU preflight (depot `julia-depot-login`) that checks files, datasets, shapes, factory assets and topology, and calls `sbatch` only after `Sanity check OK`. The job then recreates a disposable Julia project (depot `julia-depot-prod`), writes MPI preferences, builds `libwitt_ffno.so` natively and prints `FFNO_OLIVIA_JULIA_ENVIRONMENT_READY`. Home has no storage, so depots live under the project.

18.3 eagle7

Six NVIDIA T4s without Slurm. Source `~/loadnvidia.sh`, use `~/nvidiaenv/bin/python`, and set `FFNO_FSDP_LAUNCHER=.../scripts/ffno_fsdp_service_local.sh`, `FFNO_GPU_CONTROL_HOST=127.0.0.1`. Full fine-tuning runs out of memory on T4s. A 5×5 grid fits at most 2–3 GPUs, because of the rFFT bin constraint. See `response_run/falc/run_falc_eagle7.sh`.

18.4 hyades

CPU only. Source `~/loadgcc.sh`. Use node-local Julia at `/tmp/julia-1.12.3` where present, since Julia on stornext intermittently fails with "failed to map segment". System OpenMPI needs `libhcoll.so.1`, which is on hyades2, hyades3 and eagle8 only; `CXX=g++` builds the Wittmann test library. Use one CPU type per MPI job, and pin the MPICH interface.

18.5 Reproducing the FAL-C comparison

```
python response_run/falc/build_falc_cmp.py     # inputs and RH run directories
bash response_run/falc/run_cmp_rh.sh           # hyades2
bash response_run/falc/run_cmp_ffno.sh         # eagle7
python figures/falc/plot_falc_cmp.py
```

19. Tests and validation

Entry	Coverage
`julia --project=. -e 'using Pkg; Pkg.test()'`	`test/runtests.jl`: types, nodes, parallel runtime, I/O, Wittmann and continuum, population backends, mesh, synthesis, Muspel integration, HSEMHS equilibrium, solvers and gradients, FSDP protocol, correction mapping, Olivia scripts, viewing angle, log m and given atmospheres, Muspel continuum. No GPU, using mock populations
`test/mpi_*_worker.jl`	Multi-rank workers, driven by validation scripts
`scripts/validate_phase4_topology.jl`, `validate_phase5_mpi.jl`, `validate_phase6_mpi.jl`	One rank against four; restart across layouts
`scripts/validate_eos_consistency.jl`	HSEMHS table against Wittmann library
`scripts/validate_muspel_reference.jl`	Synthesis against a Muspel reference
`scripts/compare_service_with_ffnonet.py`	Service against a direct prediction; should match to float32 round-off
`scripts/run_olivia_runtime_tests.sbatch`	Regression chain on Olivia, including GPU probes, deliberate stalls and recovery
`tests/`	Python: loss, spacing, checkpoints, launcher, Wittmann, Olivia outcomes
`FFNOInversion.jl/test/ffno_jvp_cpu_check.py`	Distributed JVP against finite differences on CPU

20. UQ tooling

Script	Purpose
`scripts/uq_predict.py`	Full-domain or tiled prediction with MC dropout, `--route`, `--z-range` crops
`scripts/uq_error_atlas.py`	Error by regime and height against truth
`scripts/uq_support_distance.py`	Distance of each voxel to training data, self-matches excluded
`scripts/uq_missing_regimes.py`	Regression tree of error on input descriptors
`scripts/uq_cross_snapshot.py`	Per-level error tables
`scripts/uq_verify_alignment.py`	Orientation, grid, unit and level-order checks
`scripts/uq_retrain_setup.py`, `uq/retrain_run.sh`, `uq/olivia_finetune.sbatch`	A/B/C retraining experiment
`scripts/uq_synth.jl`, `uq_spectral_errors.py`	Spectra from predicted populations

Report: `docs/ffno3d-uq-report.md`. Its controlled fine-tuning section is still marked pending.

21. Known issues found while writing this document

These come from reading the source. Unless a measurement is mentioned, they have not been checked by experiment, and none has been changed.

Issue	Detail
Non-periodic padding in `fuse`	`models/ffno_model.py`: the depthwise $3 \times 3 \times 3$ convolution uses zero padding, and `_HaloExchangeH` fills the outer halos with zeros, though the simulations are periodic. Measured with one real column repeated on a square grid (`response_run/diagnostics_edge_test.py`, H model): outputs differ from the centre by up to 0.49 dex at the edge, 0.10 dex one pixel in, 0.017 dex at 4 pixels and 0.001 dex at 9, on 21×21. On 5×5 the first ring differs by 0.14 dex. Likely source of the 4% spread in FAL-C spectra. A circular padding adds no weights, but the trained weights assumed zero padding, so it would call for retraining or fine-tuning. The package README still says this cause is uninvestigated.
Scale weights have no effect	`train_utils.compute_loss`: `(loss*weight).sum()/(weight.sum()+1e-12)` with scalar `loss` from `WeightedMSE_L1` returns `loss`. So `weights` in the dataset, and the reweighting in `TRAINSELECT`, do not change the optimization. Coarse scales are therefore weighted by their patch counts (121 at $s = 1$, 1 at $s = 6$).
H/W spacing swap	`SpectralConv2dFull`: `ky` is built from axis H with `d=dy` and `kx` from W with `d=dx`, but $H = n_x$ and $W = n_y$ in the data and in the service. Harmless when $\Delta x = \Delta y$, as in all training data. Anisotropic grids, such as deprojected footprints at $\mu < 1$ on $\log \tau$, get swapped spacings in the frequency features.
`reject_trial` is not implemented in the objective	`Solvers.evaluate_objective!` throws on an invalid equilibrium, with the comment "the reject-trial policy arrives in Phase 4", although the README says the solver rejects that trial.
Stale documents	`docs/paper_draft_ffno_nlte.md` says the target is $\log_{10} b_i$; the code predicts $\log_{10} n_i$. `docs/ffno_full_forward.*` shows checkpointing conditioned on training; it is conditioned on gradient mode. The `logb` variable names in the loss are historical.
Inert loss cost	Source term computed each step with weight 0 (`lam_S=0`).

Issue	Detail
Unused table in the line model	`build_muspel_line_model` still builds a continuum table from FFNO hydrogen when `include_continuum=false`. It is not read, so this only costs setup time. Its default remains `include_continuum=true` for direct callers.
Region PSF	`psf_type`, `psf_file` are validated but no per-region PSF is applied.
Composition differs	Wittmann mass per H is $1.398\,m_{\mathrm{H}}$; UQ measured $\rho/(n_{\mathrm{H}}m_{\mathrm{H}}) = 1.357$ in the MULTI3D data.

22. Limits

Stokes I and non-PRD only. `model = "mhs"` rejected until HSEMHS-integration Phase 4. Equilibrium derivatives by finite differences. FFNO lines limited to Hα and Ca II 854.2 nm. The service needs NVIDIA GPUs and at least two GPU ranks. One viewing angle per run. No calibrated uncertainty. Every labelled snapshot has been used in development, so there is no pristine test set.